

16 | 云原生安全：不可忽视的关键因素

潘野 · 云原生基础架构实战课



你好，我是潘野。

从今天开始我们进入安全章节，很多同学可能有点疑惑，我们不是讲基础架构自动化的吗？这与安全有多少关联呢？

其实关联还是挺高的，因为在云原生体系中，安全就像盖房子时的地基，如果地基不牢固，房子就很容易倒塌。所以，我们需要格外地重视安全。学完今天的内容，相信你会对云原生的安全体系建立更深刻的认识。

云原生安全的特点

我们先来分析一下云原生的安全问题有什么特点。

首先，云原生应用的攻击面更大。传统的应用通常位于一个数据中心内，以往我们做好单数据中心的安全措施基本可以抵挡住 90% 的攻击。而云原生应用可能分布在多个数据中心甚至多

个云上，那么攻击者自然更容易找到攻击目标，所以这就提升了我们防护的难度。

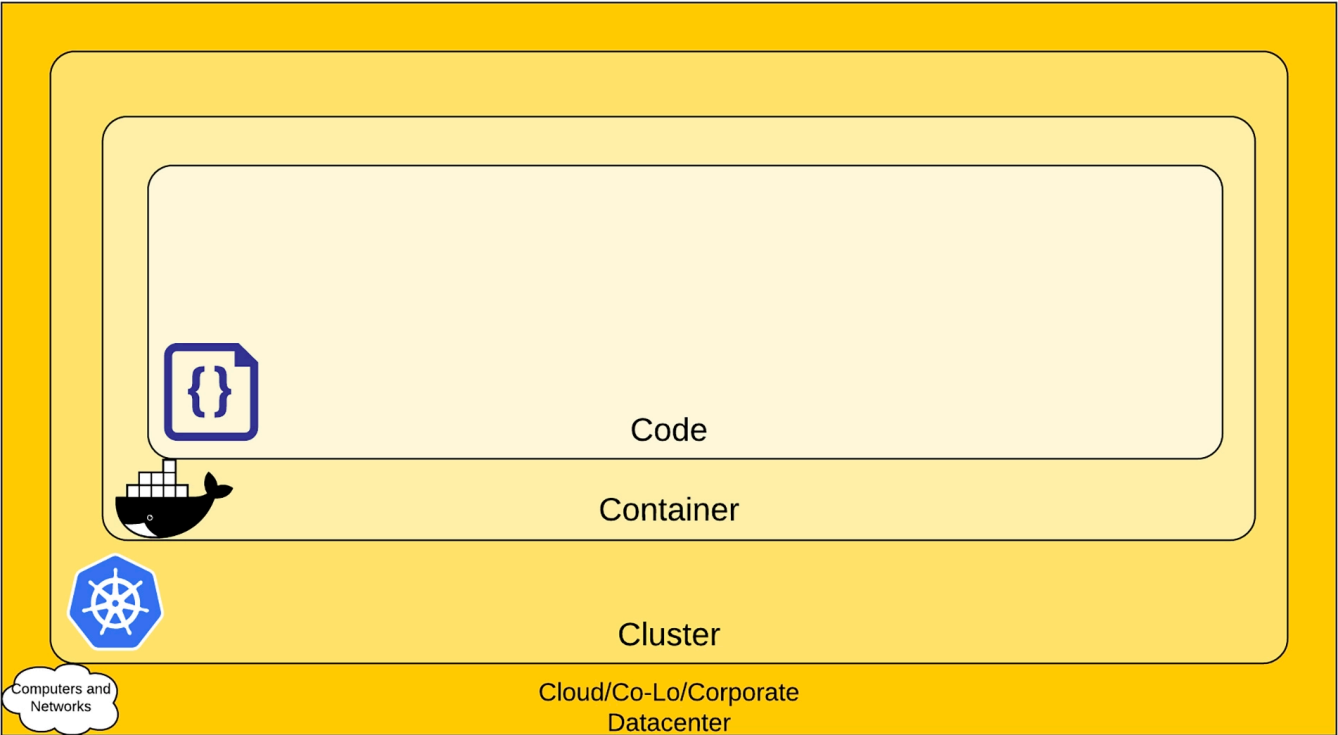
其次，云原生应用的组件通常是动态创建和销毁的，这使得传统的、基于静态配置的安全防护措施难以跟上。

然后，云原生应用通常使用容器和微服务等技术。这些技术虽然带来了很多好处，但也带来了新的安全挑战。传统的应用通常是一个整体，攻击者需要找到应用的入口才能进行攻击。而云原生应用，攻击者甚至可能从容器的漏洞发起攻击。

云原生安全的 4 个 C

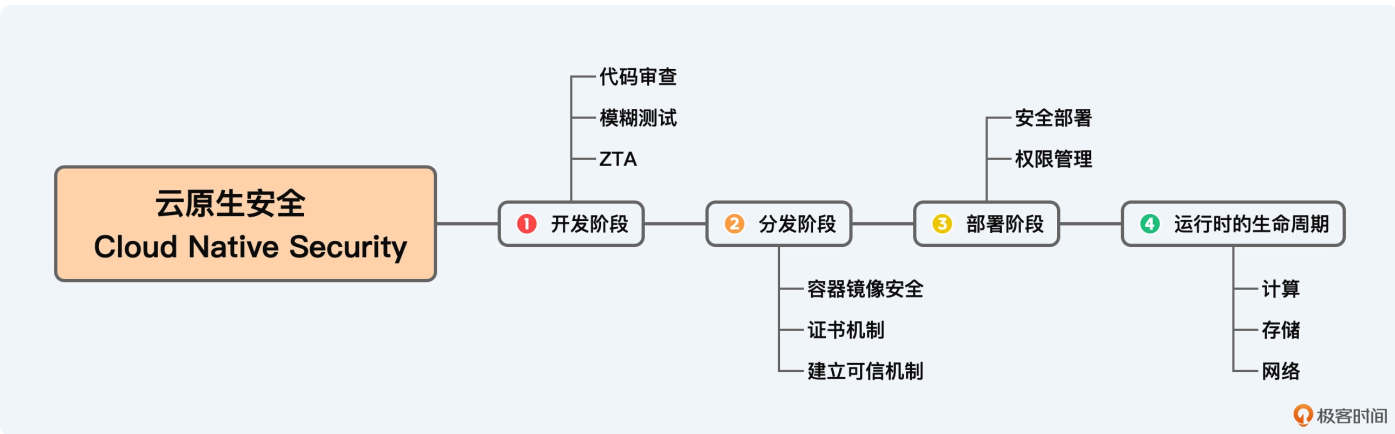
下面我们来看看云原生安全防护的思路。

Kubernetes 官方文档里曾经提出 4 个 C 安全防护的思路，分别是 Cloud、Cluster、Container 和 Code。下图来自 Kubernetes 官方文档，图里很好地解释了这四个 C 的层级是由外到内、分层来解决安全问题的。



而在最新的 Kubernetes [官方文档](#)中，从应用开发的角度重新定义了云原生安全的层级，改变了从外到内层层包裹的体系，更加关注整个应用生命周期中的安全问题。

从应用生命周期的角度来看，一共分成了四个阶段，分别是开发阶段、部署阶段、分发阶段，运行时的生命周期。这四个阶段各有各的关注点，我不同阶段对应的关注点做成了思维导图，供你参考。



无论是过去的 4C 安全体系，还是现在的应用四阶段的安全体系。在基础架构中，我们着重关注的是 **Cloud、Cluster 和 Container 三个部分**，对应到四个阶段中，主要是分发阶段、部署阶段、和运行时的生命周期三个阶段。不过需要注意的是，它们之间不是一一对应，而是一种复杂的重合关系。

接下来，我会基于 4C 体系将这些内容拆开来，逐一为你详细讲解，同时将分发阶段、部署阶段，还有运行时生命周期三个阶段与之一一对应

Cloud

云安全由云提供商和用户共同承担的，这里的用户不仅仅是公司或者团队，还包括部署在云上的应用。那么云提供商自然就要负责提供基础设施安全和平台安全，这其中包括物理安全、网络安全、数据安全和应用程序安全等。而用户负责应用安全和数据安全，包括身份验证和授权、数据加密、访问控制、安全监控等。

一般来说，你可以在云提供商的产品页面里找到关于安全产品的介绍与文档，我们以 AWS 为例子，AWS 的 [安全文档](#)中涵盖四个主题，安全基础知识、安全服务、安全最佳实践和安全

案例。

这些主题能够帮助我们了解 AWS 安全架构、安全服务和安全最佳实践，但是内容非常多，我们如何抓住其中的重点呢？

结合我的经验来看，建议你着重了解以下几个方面，并且要能做到熟练使用。

身份和访问管理 (IAM)：了解如何使用 IAM 控制对 AWS 资源的访问。

加密：了解 AWS 提供的各种加密服务，例如 Amazon S3 加密、Amazon EBS 加密和 AWS KMS。

网络安全：了解 AWS 提供的网络安全服务，例如 Amazon VPC、AWS Firewall Manager 和 AWS WAF。

数据安全：了解 AWS 提供的数据安全服务，例如 Amazon CloudTrail、AWS CloudWatch 和 AWS Config。

合规性：了解 AWS 如何帮助您满足各种合规性要求，例如 GDPR 和 PCI DSS。

对于其他的云提供商，其实主体思路也大致相同，重点要关注各个云厂商在身份管理、加密、网络安全、数据安全以及合规性等方面的文档。

那有同学一定会问了，公司是自建机房或者自建的私有云，没有公有云安全那么全的功能，又该如何应对呢？

其实在私有云或者自建机房中依然是针对身份管理、加密、网络安全、数据安全还有合规性这五个方面来实施，有很多商业或者开源安全软件可以帮你解决这五个方向的安全问题。

Cluster

谈到集群安全，Kubernetes 在设计之初就仔细考虑过安全问题。Kubernetes 的安全分成控制平面安全与数据平面的安全，那么集群安全几乎等同于前面提到的运行时生命周期。

控制平面安全

那么在 Kubernetes 控制平面的安全主要是由 apiserver 来完成的。简单来说，当对集群发起一个操作请求，在 apiserver 的内部有三个步骤，分别是认证、鉴权和准入机制。apiserver 会先验证请求者的合法性，接下来判断对请求的资源是否有操作权限，最后准入机制会根据情况修改或拒绝请求。

从 Kubernetes 1.21 开始，Kubernetes 加强了控制平面的安全功能，推荐使用 OPA（Open Policy Agent）与 Kubernetes API 服务器集成，可以对所有的 kubernetes 资源在创建、更新或删除 Kubernetes 资源之前进行授权和准入控制。

数据平面安全

数据平面的安全主要集中在两个关键方面：网络隔离和数据加密。

Kubernetes 提供了 NetworkPolicy 和 Service Mesh 两种机制来加强网络隔离。

NetworkPolicy 可以控制 Pod 之间的网络访问，以限制攻击面。

Service Mesh，例如 Istio，通过提供 MTLS（双向 TLS）功能，为 Pod 间的通信实现加密和身份验证，从而进一步增强安全性。

Kubernetes 提供了多种数据加密措施，以保护敏感数据安全。这些措施包括 Secrets 加密、使用 CSI 驱动程序加密存储卷，以及使用 Vault 等工具管理密钥。这个我们在后面的章节再详细讲解。

在网络层面，云原生体系采用 TLS 加密技术贯穿整个交互过程，确保网络流量的安全。其中 TLS 证书的管理是非常重要的一环。

Kubernetes 使用多种类型的证书，主要是两类证书，一类是服务端 / 客户端证书，用于验证服务端客户端身份；一类是 CA 证书，用于签发证书。

我们可以将 Kubernetes 集群看成一个公司，证书就像员工的工牌。工牌可以证明员工的身份，并允许他们进入公司。CA 证书就是公司的安保部门负责颁发工牌。安保部门需要确保工牌是真实有效的，并且不会被滥用。

容器 runtime 安全

容器 runtime 安全是指在容器运行期间对其进行保护，防止各种潜在的安全威胁。运行时安全对于容器来说至关重要，因为在这个阶段，容器面临着诸多安全风险，如代码注入、资源滥用、逃逸攻击等。一旦攻击者在容器运行时获得控制权，他们可以轻易地操纵应用，造成数据泄露、服务中断等严重后果。

容器运行时安全，常见的错误做法包括后面这四类。

使用不安全的镜像：镜像是容器的基础，如果镜像本身存在安全漏洞或恶意代码，那么容器运行时也会受到威胁。因此，在使用镜像之前，应仔细检查其安全性，确保其来自可靠的来源。

以 root 用户运行容器：容器默认情况下是以 root 用户运行的，这会导致容器拥有过大的权限，从而增加攻击者利用漏洞获取控制权的风险。因此，应尽量避免以 root 用户运行容器，并为容器设置最小特权。

未启用安全功能：大多数容器 runtime 都提供了丰富的安全功能。这些功能可以有效地限制容器的行为，降低安全风险。因此，我们应尽可能启用这些安全功能。

忽略运行时安全监控：容器 runtime 安全监控可以帮助我们及时发现容器中的异常行为，并采取措施阻止攻击。因此，我们应建立完善的运行时安全监控机制，并定期进行安全检查。

根据上面这些错误的做法，我们不难得出一些容器 runtime 安全最佳实践。

首先是使用安全可靠的镜像。容器文件系统应只包含必要的应用程序文件和数据，我们可以将镜像的管理贯穿在应用的分发阶段，通过 DevOps 的 CI 流水线对镜像做到自动化扫描，将不可靠的镜像排除在环境之外。

其次，容器应以最小特权原则运行，即以非 root 用户身份执行，并仅配备完成其任务所需的最低权限。

最后，Seccomp、Cgroups、SELinux 等容器运行时安全功能可以增强安全性。具体来说，Seccomp 限制容器可执行的系统调用以防止代码执行漏洞；Cgroups 控制资源使用，如

CPU、内存和磁盘，以避免资源滥用；而 SELinux 则强制容器在特定安全策略下运行，进一步增强安全护。

通过遵循这些最佳实践，可以有效地提升容器 runtime 安全性，降低安全风险。

总结

今天，我们探讨了 Kubernetes 官方对云原生安全体系的定义。这一定义现在已经从基于基础设施的 4C 层级模型，转变为以应用为核心。这表明安全策略需要覆盖应用的整个生命周期，从开发、部署到运行。

尽管这种定义上的转变，技术层面的变化并不显著，但这种趋势仍然值得我们多加关注。

在开发阶段，包括代码审查、模糊测试和零信任架构（ZTA）等安全措施，已被多数云服务提供商纳入其产品中，这对应于 4C 模型中的“Cloud”和“Code”。

而分发阶段、部署阶段、运行时的生命周期这三个阶段中的安全措施，贯穿在 4C 中 Cluster 和 container 的管理中。例如容器镜像的安全、容器权限的控制、集群权限管理、应用数据加密，网络隔离这些手段，都是在 Cluster 和 container 级别完成的。

在接下来的章节中，我们会重点介绍我们该如何将云原生安全措施落地，确保它们有效地提供安全保护，真正发挥防火作用。

思考题

在 Kubernetes 集群和容器级别，具体有哪些安全软件来完成安全防护？

欢迎在评论区与我讨论。如果这一讲对你有启发，也欢迎分享给身边更多朋友。

AI智能总结

1. 云原生应用的特点包括攻击面更大、组件动态创建销毁、使用容器和微服务技术，带来新的安全挑战。
2. 云原生安全防护思路包括Cloud、Cluster、Container和Code，以及应用生命周期的四个阶段：开发、部署、分发和运行时。
3. Kubernetes在设计中考虑了控制平面安全和数据平面安全，包括认证、鉴权、准入机制、网络隔离和数据加密等措施。

4. 容器runtime安全的最佳实践包括使用安全可靠的镜像、以最小特权原则运行容器、启用容器运行时安全功能。
5. 云原生安全体系的定义已从基于基础设施的4C层级模型转变为以应用为核心，安全策略需覆盖应用的整个生命周期，从开发、部署到运行。
6. 在开发阶段，安全措施包括代码审查、模糊测试和零信任架构（ZTA）等。
7. 分发阶段、部署阶段、运行时的生命周期中的安全措施，贯穿在Cluster和container的管理中，包括容器镜像的安全、容器权限的控制、集群权限管理、应用数据加密、网络隔离等手段。
8. Kubernetes集群和容器级别的安全防护可以借助安全软件来完成，具体有哪些安全软件需要进一步讨论。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (2)

最新 精选



Geek_45a572

2024-05-01 来自陕西

Apiserver不暴露公网，放在私有子网中，pod中通过nat网关出去。再加上node和lb的安全组。集群应该就很安全吧。 加这么多审核的组件性能也有损耗。

作者回复: Hi，同学你好

有的时候，攻击不一定来自外部，有时候也有来自内部的攻击。

此外，无论采用了多少种防护措施，总之会有一个endpoint要暴露出去，比如应用要对公网发布，ingress-nginx上总要建一个面向公网的ELB，但是ingress-nginx本身的也可能会出现一些漏洞，这就给外部攻击带来可乘之机。



meta-algorithmX

2024-04-29 来自北京

集群：

1. 使用CIS Benchmarks审查组件安全配置（常见软件 kube-Bench）
2. 宿主机Node 操作系统内核安全增强工具（AppArmor， seccomp）

容器：

1. 容器镜像安全扫描 (Clair, Trivy)
2. 容器运行时runtimeClass安全 (Kata, gVisor)
3. 静态分析工作负载 (kubernetes 资源及docker file) (kubesecc)
4. 容器内syscall行为监控 (Strace, Falco)

当然以上只是常见的第三方软件工具，kubernetes自身也有许多保证安全的组件和机制，比如RBAC，PodSecurityPolicy，Audit Logging，利用OPA及admission机制保证供应链安全等等。

